

KIT DE SOUTENANCE — MISSION DEV — UpcycleConnect

Généré à partir de l'analyse du code réel du projet.

01 — Architecture du projet (UpcycleConnect)

Projet : **UpcycleConnect** (dépôt `renova-systems-projet-annuel`) — plateforme d'économie circulaire (dons/ventes d'objets, conteneurs/casiers connectés, espace pro, back-office admin).

Stack technique (réel, vérifié dans le code)

Couche	Techno	Preuve (fichier)
Backend	Go (net/http standard, pas de framework)	API/main.go , API/go.mod
Base de données	MySQL 8	docker-compose.yml (image: mysql:8.0), API/bdd/db.go
Frontend	HTML / CSS / JS vanilla (pas de framework) servi par Nginx	Frontend/*.html , Frontend/script/*.js , Frontend/nginx.conf
Conteneurisation	Docker + Docker Compose	docker-compose.yml , docker-compose-prod.yml , API/Dockerfile , Frontend/Dockerfile
Auth	JWT (HS256)	API/auth/jwt.go
Paieement	Stripe (Checkout + Connect)	API/admin/stripe.go , API/admin/webhook.go
Notifications push	OneSignal	API/admin/notifications.go

Structure générale

```
renova-systems-projet-annuel/
├── API/                               ← BACKEND Go (port 8081)
│   ├── main.go                       ← point d'entrée : enregistre toutes les routes + ListenAndServe(":8081")
│   ├── go.mod / go.sum               ← dépendances Go
│   └── Dockerfile                    ← build multi-stage du backend
│   ├── auth/
│   │   └── jwt.go                    ← génération/vérification JWT + middlewares de rôle
│   ├── bdd/
│   │   ├── db.go                    ← connexion MySQL (NewDB)
│   │   └── *Req.go                   ← requêtes par domaine (userReq, annonceReq, boxReq...)
│   ├── models/                      ← structs Go (Annonce, User, Evenement...)
│   ├── route/                       ← déclaration des routes HTTP (une fonction Routes* par domaine)
│   └── admin/                        ← handlers HTTP (la logique des endpoints) + upload + pdf + stripe + webhook
├── Frontend/                         ← FRONTEND statique (servi par Nginx)
│   ├── *.html                       ← pages (login, espClient, espPro, admin_*, salarie/*, annonceAll...)
│   ├── script/                      ← JS (config.js, login.js, dashClient.js, admin/*, salarie/*)
│   ├── style/                       ← CSS
│   ├── assets/                      ← logo, favicon (logoUpcycle.png / .ico)
│   ├── nginx.conf                   ← config Nginx (proxy /api, /uploads, URLs propres)
│   └── Dockerfile                   ← build du conteneur frontend (nginx)
├── db/
│   └── init.sql                     ← schéma + données MySQL (import auto au 1er démarrage Docker)
├── docker-compose.yml                ← stack LOCALE (build local : uc_mysql / uc_backend / uc_frontend)
└── docker-compose-prod.yml           ← stack VM/PROD (images Docker Hub amelbdj/upcycle-*)
```

```
└─ .env.example      ← modèle de variables d'environnement
└─ pa2026.sql        ← dump SQL de référence
```

Où se trouve quoi (chemins exacts)

Élément	Chemin
Frontend	Frontend/
Backend / API	API/ (entrée : API/main.go)
Config Docker (local)	docker-compose.yml
Config Docker (prod/VM)	docker-compose-prod.yml
Dockerfile backend	API/Dockerfile
Dockerfile frontend	Frontend/Dockerfile
Config Nginx	Frontend/nginx.conf
Base de données (schéma + seed)	db/init.sql (et pa2026.sql de référence)
Variables d'environnement	.env.example (à copier en .env), injectées via docker-compose*.yml
Connexion DB (code)	API/bdd/db.go
Config API côté front	Frontend/script/config.js

Comment le front communique avec l'API

- Frontend/script/config.js définit **API_BASE_URL** automatiquement :
- WAMP local** (localhost + URL contenant /Frontend/) → http://localhost:8081 (backend Go en direct)
- Docker / VM** (sinon) → origin + "/api" (ex : https://upcycleconnect.pro/api)
- Tous les appels JS font fetch(\ \${API_BASE_URL}/...) avec un header Authorization: Bearer (token en localStorage).
- En Docker/VM, Nginx (Frontend/nginx.conf) reçoit /api/... , réécrit ^/api/(.*)\$ /\$1 et **proxy_pass** vers http://backend:8081 .

```
Navigateur — fetch /api/admin/users —> Nginx (frontend:80)
└─ rewrite /api/(.*)$ /$1 —> proxy_pass http://backend:8081/admin/users —> Go
```

Comment l'API communique avec la base

- API/bdd/db.go → NewDB() ouvre la connexion MySQL avec le DSN user:pass@tcp(host:port)/pa2026?parseTime=true&charset=utf8mb4 .
- Les identifiants viennent des **variables d'environnement** (DB_HOST , DB_PORT , DB_USER , DB_PASS , DB_NAME) ; en local, valeurs par défaut localhost/3306/root/root/pa2026 .
- La variable globale bdd.Db (*sql.DB) est utilisée par tous les fichiers API/bdd/*Req.go via Db.Query(...) , Db.QueryRow(...) , Db.Exec(...) .
- En Docker, docker-compose.yml fournit DB_HOST: mysql (nom du service MySQL dans le réseau uc_net) .

Schéma d'ensemble

```
[Navigateur]
├─ HTML/JS statiques + fetch API (Bearer JWT)
└─
[ Nginx frontend:80 ] — / , /login, /admin ... → fichiers .html (URLs propres)
├─ /api/* → proxy → backend:8081
├─ /uploads/* → proxy → backend:8081/uploads (images/PDF)
└─
[Go backend:8081] — auth JWT (middlewares) — handlers (API/admin/*) — bdd (API/bdd/*Req.go)
└─
[MySQL mysql:3306] base `pa2026`
```

02 — Carte des fonctionnalités

Légende criticité : ● = à maîtriser absolument · ● = important · ● = secondaire.

Authentification ●

- **Rôle** : tous.
- **Front** : `Frontend/login.html`, `Frontend/script/login.js`, `Frontend/register.html`, `Frontend/script/register.js`, `Frontend/script/config.js` (token en `localStorage`), `Frontend/script/auth_guard.js` (garde de page).
- **Backend** : `API/admin/users.go` (Login, Inscription), `API/auth/jwt.go` (JWT + middlewares).
- **Tables** : `utilisateur`, `log_connexion`.
- **Endpoints** : `POST /admin/login`, `POST /auth/inscription`, `POST /auth/check-email`, `POST /auth/forgot-password`, `POST /auth/reset-password`.

Gestion des utilisateurs ●

- **Rôle** : Administrateur.
- **Front** : `Frontend/admin_users.html`, `Frontend/script/admin/user.js` (pagination 10/page).
- **Backend** : `API/admin/users.go`, `API/bdd/userReq.go`.
- **Tables** : `utilisateur`, `documents_legaux`.
- **Endpoints** : `GET /admin/users`, `GET /admin/users/{id}`, `GET /admin/users/search`, `GET /admin/users/role/{role}`, `POST /admin/users/add`, `PUT /admin/users/modify/{id}`, `DELETE /admin/users/delete/{id}`, `PUT /admin/users/validate|refuse|ban/{id}`.

Rôles & permissions ●

- **Rôles** (enum `utilisateur.role`) : `Utilisateur`, `Pro`, `Salarié`, `Administrateur`.
- **Backend** : `API/auth/jwt.go` → `VerifyTokenMiddleware` (vérifie le JWT) et `VerifyRoleMiddleware(next, roles...)` (vérifie le rôle). Rôle mis dans `context` (`userRole`).
- **Front** : redirection par rôle dans `Frontend/script/login.js` ; garde côté page `Frontend/script/auth_guard.js` (`checkSession(role)`).

Annonces (dons / ventes) ●

- **Rôle** : Utilisateur (crée), Pro (achète/sponsorise), Admin (valide).
- **Front** : `Frontend/annonceAll.html` + `Frontend/script/annonces/annonceAll.js`, `Frontend/oneAnnonce.html` + `oneAnnonce.js`, `Frontend/espClient.html` + `Frontend/script/dashClient.js`, admin : `Frontend/script/admin/annonce.js`.
- **Backend** : `API/admin/annonce.go`, `API/bdd/annonceReq.go`.
- **Tables** : `annonce`, `categorie`.
- **Endpoints** : `GET /api/annonces`, `GET /api/annonces/all`, `GET /mes-annonces`, `POST /admin/annonces/add`, `PUT /admin/annonces/modify/{id}`, `PUT /admin/annonces/validate|refuse/{id}`, `DELETE /admin/annonces/delete/{id}`, `POST /api/annonces/vendre`, `POST /api/payment-annonce`.

Catégories / matériaux ●

- **Rôle** : Admin (gère), tous (filtre).
- **Front** : filtre dynamique `Frontend/script/annonces/annonceAll.js` (`buildCategoryFilter`).
- **Backend** : `API/admin/categorie.go`, `API/bdd/categorieReq.go`.
- **Table** : `categorie` (libellés : Textile, Bois, Plastique, Métal).
- **Endpoints** : `GET /admin/categories`, `POST /admin/categories/add`, `DELETE /admin/categories/delete/{id}`.

Conteneurs & casiers (logistique) ●

- **Rôle** : Admin (gère), Utilisateur (dépose/récupère), Pro (récupère).
- **Front** : `Frontend/admin_conteneurs.html` + `Frontend/script/admin/box.js` (+ **rapport logistique PDF** via jsPDF).
- **Backend** : `API/admin/box.go`, `API/bdd/boxReq.go`.
- **Tables** : `conteneur`, `box`, `box_conteneur`, `depot_box`, `historique_conteneurs`.

- **Endpoints** : `GET /api/admin/conteneurs` , `GET /api/admin/conteneur/{id}/boxes` , `POST /api/admin/conteneur/create` , `POST /api/admin/box/add` , `PUT /api/admin/box/update` .

Réservation / dépôt / retrait (hardware simulé) ●

- **Front** : `Frontend/simulateur.html` (simulateur dépôt/retrait) , `Frontend/script/dashClient.js` (boxes user).
- **Backend** : `API/admin/box.go` (deposit/collect/reserve).
- **Tables** : `depot_box` , `historique_conteneurs` , `box` .
- **Endpoints** : `POST /api/box/reserve` , `POST /api/box/deposit` , `POST /api/box/collect` , `POST /api/boxes/valider-retrait` , `POST /api/hardware/simulate-deposit` , `POST /api/hardware/simulate-withdrawal` , `GET /api/user/boxes` , `GET /api/user/pickups/{id}` .

Commandes / transactions / commissions ●

- **Rôle** : Utilisateur (achète), Admin (suivi finances).
- **Front** : `Frontend/admin_finances.html` + `Frontend/script/admin/finance.js` .
- **Backend** : `API/admin/orderReq.go` , `API/admin/stripe.go` , `API/bdd/order.go` .
- **Tables** : `order` , `paiement` , `document` (factures).
- **Commission** : **5 %** — voir `API/admin/stripe.go` ($(unitAmount * 5) / 100 / prix * 0.05$).
- **Endpoints** : `GET /admin/finance/overview` , `GET /admin/finance/transactions` , `GET /api/user/payment-history` , `GET /api/user/achats` .

Upcycling Score (impact éco) ●

- **Rôle** : Utilisateur.
- **Front** : `Frontend/espClient.html` + `Frontend/script/dashClient.js` (`loadEcoScore`).
- **Backend** : `API/admin/annonce.go` (`GetEcoStatsHandler`), `API/bdd/annonceReq.go` (`GetUserEcoStats`), `API/bdd/boxReq.go` (`CalculateAndAddScore`).
- **Table** : `utilisateur.score` , `upcycling_score` .
- **Endpoints** : `GET /api/user/stats` , `GET /api/user/ecostats` .
- **Calcul** : $gainScore = poids_kg \times coefficient(matériau)$ — coefficients dans `CalculateAndAddScore` (textile 15, métal 10, plastique 8, bois 5, autre 3). ⚠️ **À confirmer** : dans le code actuel la variable `matériau` n'est pas remplie par la requête SQL → le coefficient tombe toujours sur `autre` (3.0). (Point d'amélioration honnête à connaître.)

Événements & formations ●

- **Rôle** : Salarié (crée), Utilisateur/Pro (s'inscrit, paie), Admin (valide).
- **Front** : `Frontend/evenement.html` + `Frontend/script/affichageEvt.js` , salarié : `Frontend/salarie/salarie_events.html` + `Frontend/script/salarie/event.js` , planning : `Frontend/script/salarie/planning.js` .
- **Backend** : `API/admin/event.go` , `API/bdd/eventReq.go` , `API/admin/stripe.go` (`CreateEventCheckoutSession`).
- **Tables** : `evenement` , `inscription` , `ressource_pedagogique` .
- **Endpoints** : `GET /admin/evenements` , `POST /admin/evenements/add` , `PUT /admin/evenements/validate|refuse/{id}` , `POST /admin/evenements/inscription|desinscription` , `POST /api/web/checkout/evenement` , `GET /admin/evenements/inscrits/{id}` , `GET /admin/evenements/ressources/{id}` .
- **Règle métier** : un salarié doit avoir un `stripe_account_id` pour déposer un event (`API/admin/event.go` , `CreateEvenement`).

Articles / actualités ●

- **Rôle** : Salarié (rédige), Admin (valide), tous (lecture).
- **Front** : `Frontend/article.html` + `Frontend/script/affichageArt.js` , salarié : `Frontend/script/salarie/article.js` .
- **Backend** : `API/admin/article.go` , `API/bdd/articlesReq.go` .
- **Table** : `article_news` (PK `id_article`).
- **Endpoints** : `GET /admin/articles` , `POST /admin/articles/add/{action}` , `PUT /admin/articles/validate|refuse/{id}` , `GET /admin/articles/salarie/{id}` .

Forum ●

- **Front** : `Frontend/forum.html` + `Frontend/script/forum.js` , salarié : `Frontend/script/salarie/forum.js` .

- **Backend** : `API/admin/forum.go` , `API/bdd/forumReq.go` .
- **Tables** : `topic_forum` , `message_forum` .
- **Endpoints** : `GET/POST /user/forums` , `GET/POST /user/forums/messages` , `GET /admin/forum/messages` , `PUT /admin/forum/messages/moderate/{id}` , `GET /admin/forum/stats` .

Messagerie / chat (WebSocket) ●

- **Front** : `Frontend/script/messagerie/messagerie.js` .
- **Backend** : `API/admin/chat.go` , `API/bdd/chatReq.go` .
- **Table** : `message` .
- **Endpoints** : `GET /ws/chat` (WebSocket), `GET /api/chat/conversations` , `GET /api/chat/history` .

Espace Pro (abonnements, projets, sponsoring) ●

- **Front** : `Frontend/espPro.html` + `Frontend/script/dashPro.js` .
- **Backend** : `API/route/pro.go` , `API/admin/stripe.go` , `API/admin/projet.go` , `API/admin/etape.go` .
- **Tables** : `abonnement` , `plan_abo` , `projet_pro` , `etapes_projet` .
- **Endpoints** : `POST /api/pro/subscribe|upgrade|cancel|portal` , `GET /api/pro/sync|invoices|projets|etapes` , `POST /api/pro/projets/create` , `POST /api/pro/annonces/sponsor` .

Back-office admin ●

- **Front** : `Frontend/admin_dashboard.html` (+ `Frontend/script/admin/dash.js`), `admin_users.html` , `admin_validations.html` , `admin_finances.html` , `admin_documents.html` , `admin_conteneurs.html` .
- **Backend** : dossier `API/admin/` .
- **Sécurité** : toutes les routes `GET /admin/*` (hors login) sont protégées par `VerifyTokenMiddleware` / `VerifyRoleMiddleware` .

Notifications ●

- **Front** : `Frontend/script/salarie/userInfo.js` (`chargerNotifications` , cloche), SDK OneSignal inline dans `espClient.html` / `espPro.html` / `admin_*` (`OneSignal.login(userId)`).
- **Backend** : `API/admin/notifications.go` (`SendPushNotification` OneSignal, `NotifyAllAdmins` , `CreateNotification`).
- **Table** : `notification` .
- **Endpoints** : `POST /admin/notifications/send` , `GET /admin/notifications/user/{id}` , `POST /admin/notifications/user/{id}/read` .

Génération PDF ●

- **Factures / contrats** (serveur, lib `go-pdf/fpdf`) : `API/admin/pdf.go` (`GenerateInvoicePDF` , `GenerateContractPDF`) → stockés dans le volume `/app/uploads/documents/` , listés dans `Frontend/admin_documents.html` + `Frontend/script/admin/documents.js` .
- **Rapport logistique** (client, lib `jsPDF`) : `Frontend/script/admin/box.js` (`genererRapportLogistique`).
- **Table** : `document` .

Multilingue ●

- **Front** : `Frontend/script/admin/translate.js` (`t()` , `appliquerTraductions()` , sélecteur de langue en bas à droite), attributs `data-i18n` .
- **Backend** : `API/admin/translation.go` , `API/bdd/translation.go` .
- **Tables** : `translations` (`lang_code` , `msg_key` , `msg_value`), `languages` .
- **Endpoints** : `GET /api/translations?lang=fr` , `GET /api/languages` , `POST /admin/translations/add` .

Paie Stripe ●

- **Backend** : `API/admin/stripe.go` (Checkout + Connect + commission 5 %), `API/admin/webhook.go` (`POST /api/stripe/webhook`).
- **Config** : clés en variables d'env `STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET` (fallback en dur dans `stripe.go`).
- **Uploads / fichiers** : `API/admin/upload.go` (`SaveUpload` : validation ext + MIME + taille, nom UUID), route `/uploads/*` , volume `uploads_data` .

- **Local** : `docker-compose.yml` (build local).
- **Prod / VM** : `docker-compose-prod.yml` (images `amelbdj/upcycle-backend` , `amelbdj/upcycle-frontend`), derrière un Nginx hôte HTTPS (`https://upcycleconnect.pro`).

03 — Spécification API

Swagger/OpenAPI : non trouvé dans le code actuel. Cette doc est générée à partir de `API/route/*.go` et `API/admin/*.go` . Base URL locale : `http://localhost:8081` — Prod : `https://upcycleconnect.pro/api` (Nginx retire le préfixe `/api`). Auth : header `Authorization: Bearer <token>` `JWT` sur toutes les routes protégées. Toutes les routes ont un `OPTIONS` (CORS préflight).

Convention de sécurité

- **Public** : login, inscription, mot de passe oublié, listes publiques d'annonces.
- **Token requis** : `VerifyTokenMiddleware` (`API/auth/jwt.go`).
- **Rôle requis** : `VerifyRoleMiddleware(handler, "Salarié", "Administrateur")` etc.

Authentification — `API/route/auth.go` , `API/route/users.go` → `API/admin/users.go`

Méthode	Route	Rôle	Handler	Paramètres	Réponse	Erreurs
POST	<code>/admin/login</code>	public	<code>admin.Login</code>	JSON <code>{email, mot_de_passe}</code>	<code>{token, id, role, prenom, score, tutorialVu, validation}</code>	401 email/mdp incorrect
POST	<code>/auth/inscription</code>	public	<code>admin.Inscription</code>	JSON user	<code>{message}</code> 201	500 email déjà pris
POST	<code>/auth/check-email</code>	public	—	<code>{email}</code>	dispo	—
POST	<code>/auth/forgot-password</code>	public	—	<code>{email}</code>	envoi lien	—
POST	<code>/auth/reset-password</code>	public	—	<code>{token, password}</code>	ok	400 token invalide

Exemple :

```
curl -X POST http://localhost:8081/admin/login \  
-H "Content-Type: application/json" \  
-d '{"email":"admin@upcycle.fr","mot_de_passe":"motdepasse"}'
```

Utilisateurs (admin) — `API/route/users.go` → `API/admin/users.go`

Méthode	Route	Rôle	Paramètres	Réponse
GET	<code>/admin/users</code>	Salarié/Admin	—	<code>[User]</code>
GET	<code>/admin/users/{id}</code>	token	id (path)	User
GET	<code>/admin/users/search?search=</code>	token	query	<code>[User]</code>
GET	<code>/admin/users/role/{role}</code>	token	role (path)	<code>[User]</code>
POST	<code>/admin/users/add</code>	token	JSON user	<code>{message, id}</code>
PUT	<code>/admin/users/modify/{id}</code>	token	JSON	200
DELETE	<code>/admin/users/delete/{id}</code>	token	id	utilisateur suppr

Méthode	Route	Rôle	Paramètres	Réponse
PUT	/admin/users/validate/{id}	token	id	{message}
PUT	/admin/users/refuse/{id}	token	{motif}	{message}
PUT	/admin/users/ban/{id}	token	id	{message}

Exemple :

```
curl http://localhost:8081/admin/users -H "Authorization: Bearer $TOKEN"
```

Annonces — [API/route/annonces.go](#) → [API/admin/annonce.go](#)

Méthode	Route	Rôle	Paramètres	Réponse
GET	/api/annonces/all?id=	public	id user (query)	[Annonce] (jointes catégorie/user)
GET	/mes-annonces?id=	token	id user	[Annonce]
GET	/admin/annonces	token	—	[Annonce] (toutes)
GET	/admin/annonces/search?search=	token	query	[Annonce]
POST	/admin/annonces/add	token	multipart (titre, description, prix, id_categorie, image...)	201
PUT	/admin/annonces/modify/{id}	token	multipart	200
PUT	/admin/annonces/validate/{id}	token	id	{message}
PUT	/admin/annonces/refuse/{id}	token	id	{message}
DELETE	/admin/annonces/delete/{id}	token	id	{message}
POST	/api/payment-annonce	token	JSON paiement	session Stripe

Catégories — [API/route/categories.go](#) → [API/admin/categorie.go](#)

Méthode	Route	Rôle	Réponse
GET	/admin/categories	token	[[{id, libelle}]]
POST	/admin/categories/add	token	{libelle} → ok
DELETE	/admin/categories/delete/{id}	token	ok

Conteneurs / casiers — [API/route/logistique.go](#) → [API/admin/box.go](#)

Méthode	Route	Rôle	Réponse
GET	/api/admin/conteneurs	token	[Conteneur] (avec total_boxes)
GET	/api/admin/conteneur/{id}/boxes	token	[Box]
POST	/api/admin/conteneur/create	token	{nom, adresse, nombre_de_boxs}
POST	/api/admin/box/add	token	{id_conteneur, taille}
PUT	/api/admin/box/update	token	{box_id, statut}
POST	/api/box/reserve /deposit /collect	token	flux dépôt/retrait
GET	/api/user/boxes?user_id=	token	dépôts de l'utilisateur
GET	/api/user/pickups/{id}	token	retraits

Événements — [API/route/evenements.go](#) → [API/admin/event.go](#)

Méthode	Route	Rôle	Paramètres	Réponse
GET	/admin/evenements	token	—	[Evenement]
POST	/admin/evenements/add	token	multipart (titre, type, tarif, image, plan_pdf, ressources...)	ok (⚠ exige stripe_account_id du salarié)
PUT	/admin/evenements/{id}	token	multipart	modif
PUT	/admin/evenements/validate/{id} /refuse/{id}	token	id	{message}
POST	/admin/evenements/inscription /desinscription	token	{id_user, id_event}	ok
POST	/api/web/checkout/evenement	token	{id_user, id_event}	{checkout_url} (Stripe)
GET	/admin/evenements/inscrits/{id}	token	id event	liste inscrits
GET	/admin/evenements/ressources/{id}	token	id event	ressources PDF

Finance
—
API/route/finance.go
→
API/admin/orderReq.go

Méthode	Route	Rôle	Réponse
GET	/admin/finance/overview	token	{volumeMois, revenuMois}
GET	/admin/finance/transactions	token	[Transaction]
GET	/api/user/payment-history	token	historique

Eco / Score
—
API/route/divers.go
→
API/admin/annonce.go

Méthode	Route	Réponse
GET	/api/user/stats?user_id=	{score, objets_donnes, dechets_evites}
GET	/api/user/ecostats?user_id=	idem

Pro / abonnements
—
API/route/pro.go
→
API/admin/stripe.go , projet.go

Méthode	Route	Réponse
POST	/api/pro/subscribe /upgrade /cancel /portal	flux abonnement Stripe
GET	/api/pro/sync /invoices /projets /etapes	données pro
POST	/api/pro/projets/create , /api/pro/etapes/create	création
POST	/api/pro/annonces/sponsor	sponsoriser une annonce

Forum / Chat
—
API/route/forum.go , API/route/chat.go

Méthode	Route	Réponse
GET/POST	/user/forums	sujets
GET/POST	/user/forums/messages?topic_id=	messages (visibles = est_modere=0)
PUT	/admin/forum/messages/moderate/{id}	modération
GET	/ws/chat	WebSocket messagerie
GET	/api/chat/conversations /api/chat/history	chat

Notifications
—
API/route/notifications.go
→
API/admin/notifications.go

Méthode	Route	Réponse
POST	/admin/notifications/send	{nombre} envoyées
GET	/admin/notifications/user/{id}	[notif]
POST	/admin/notifications/user/{id}/read	marque lu

Traductions — [API/route/traductions.go](#)

Méthode	Route	Réponse
GET	/api/translations?lang=fr	{clé: valeur}
GET	/api/languages	[{code, name}]
POST	/admin/translations/add	import JSON

Documents / PDF — [API/route/documents.go](#) → [API/admin/document.go](#)

Méthode	Route	Réponse
GET	/admin/documents	[{type_doc, url_pdf, ...}]
—	GET /uploads/documents/...	fichier PDF (FileServer, API/route/auth.go)

Fichiers statiques / uploads — [API/route/auth.go](#)

- GET /uploads/* → http.FileServer sur UPLOAD_DIR (/app/uploads) : images annonces/articles/events/formations + PDF documents.

04 — Guide de modification en direct (anti-panique) — DÉTAILLÉ

Schéma mental du projet (à réciter) : Frontend/*.html (structure) + Frontend/script/*.js (fetch \${API_BASE_URL}/route) → **Nginx** /api → API/route/<domaine>.go (déclare la route) → API/admin/<domaine>.go (handler) → API/bdd/<domaine>Req.go (SQL) → API/models/<domaine>.go (struct).

Réflexe après CHAQUE modif : - Modif **backend (Go)** → `docker compose up -d --build backend` (≈20-40 s) puis tester. - Modif **frontend (HTML/JS/CSS)** → `WEB_PORT=8088 docker compose up -d --build frontend` **OU** ouvrir via WAMP + **Ctrl+Shift+R**. - Toujours vérifier dans **F12** → **Réseau** le code HTTP de l'appel.

0. Les 3 gestes qui sauvent

```
# Compiler l'API sans démarrer (détecte les erreurs Go en 2 s)
cd API && go build ./...

# Reconstruire + relancer un seul service
docker compose up -d --build backend      # ou frontend

# Voir l'erreur exacte
docker compose logs -f backend
```

Récupérer un token pour tester en curl (à garder sous la main) :

```
TOKEN=$(curl -s -X POST http://localhost:8081/admin/login \
-H "Content-Type: application/json" \
```

```
-d '{"email":"test.admin@renova.test","mot_de_passe":"TON_MDP"}' | grep -o '"token": "[^"]*"' | cut -d'"' -f4)
echo $TOKEN
```

A. Ajouter un champ dans un formulaire

Exemple complet : ajouter `telephone` à l'utilisateur

1. Base de données — ajouter la colonne :

```
docker exec -it uc_mysql mysql -uupcycle -pupcyclePass123 pa2026
```

```
ALTER TABLE utilisateur ADD COLUMN telephone VARCHAR(20) NULL;
```

2. Struct Go — `API/models/users.go` : ajouter dans le struct `User` :

```
Telephone string `json:"telephone"`
```

3. Requête SQL — `API/bdd/userReq.go` : ajouter `telephone` dans le `SELECT` et dans le `Scan` correspondant.

```
// AVANT
"SELECT id, nom, prenom, email, role, score FROM utilisateur WHERE id = ?"
// APRÈS
"SELECT id, nom, prenom, email, role, score, COALESCE(telephone, '') FROM utilisateur WHERE id = ?"
// et dans .Scan(...) ajouter &user.Telephone à la fin, dans le MÊME ordre
```

⚠ Règle d'or : l'ordre des colonnes du `SELECT` = l'ordre des `&champ` dans le `Scan`.

4. Handler — `API/admin/users.go` : lire le champ à la création/modif :

```
user.Telephone = r.FormValue("telephone") // si formulaire multipart
// ou déjà rempli par json.NewDecoder(r.Body).Decode(&user) si JSON
```

5. Frontend — HTML (formulaire) :

```
<input type="text" id="add-telephone" placeholder="Téléphone">
```

JS (`Frontend/script/admin/user.js`, fonction `CreateUser`) :

```
body: JSON.stringify({ nom, prenom, email, role, mot_de_passe: mdp, telephone: document.getElementById("add-telephone").value })
```

6. Rebuild + test :

```
docker compose up -d --build backend
curl -X PUT http://localhost:8081/admin/users/modify/1 -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"telephone": "0600000000"}'
# puis vérifier en base :
docker exec uc_mysql mysql -uupcycle -pupcyclePass123 pa2026 -e "SELECT telephone FROM utilisateur WHERE id=1;"
```

Variante rapide (front seulement) : ajouter un champ à une annonce

- Le formulaire d'annonce envoie un `FormData` (multipart). Ajouter :

```
formData.append("ville", document.getElementById("ville").value);
```

- Côté Go (`API/admin/annonce.go`), le lire : `ann.Ville = r.FormValue("ville")` (le champ existe déjà ici : `ville`).

B. Ajouter un filtre

Option 1 (RECO en direct) — filtre 100 % front, aucun rebuild

Sur la page annonces, les données sont déjà en mémoire dans `allAnnonces` (`Frontend/script/annonces/annonceAll.js`). Modèle existant = `toggleFilter` / `buildCategoryFilter` .

Exemple : filtrer par **ville** :

```
function filtererParVille(ville) {
  const filtered = allAnnonces.filter(
    (a) => (a.ville || "").toLowerCase() === ville.toLowerCase()
  );
  displayAnnonces(filtered);
}
```

Brancher sur un bouton : `<button onclick="filtererParVille('Paris')">Paris</button>` . **Test** : recharger `/annonces` , cliquer → seules les annonces de Paris s'affichent.

Option 2 — filtre côté serveur

API — `API/admin/annonce.go` : `ville := r.URL.Query().Get("ville")` . **DB** — `API/bdd/annonceReq.go` : ajouter la condition (garder le paramètre `?`) :

```
"SELECT ... FROM annonce WHERE UPPER(titre) LIKE ? AND (? = '' OR ville = ?)"
// args : search, ville, ville
```

Test : `curl "http://localhost:8081/admin/annonces/search?ville=Paris" -H "Authorization: Bearer $TOKEN"` .

C. Modifier une règle métier

C1. Taux de commission (5 % → 10 %)

- **Fichier** : `API/admin/stripe.go` .
- Chercher (Ctrl+F) : `* 5` / `100` et `0.05` .

```
// AVANT
commission := (unitAmount * 5) / 100
commissionEuros := prix * 0.05
// APRÈS (10 %)
commission := (unitAmount * 10) / 100
commissionEuros := prix * 0.10
```

- **Rebuild + test** : `docker compose up -d --build backend` , puis un achat → `GET /admin/finance/overview` doit refléter la nouvelle commission.

C2. Upcycling Score (coefficients)

- **Fichier** : `API/bdd/boxReq.go` , fonction `CalculateAndAddScore` .

```
coefficients := map[string]float64{
  "textile": 15.0, "metal": 10.0, "bois": 5.0, "plastique": 8.0, "autre": 3.0,
}
gainScore := poids * coef
```

- Exemple : doubler la valeur du textile → `"textile": 30.0` .
- **Test** : `GET /api/user/stats?user_id=1` avant/après un dépôt validé.
- **À savoir (honnête)** : actuellement `materiau` n'est pas relu depuis la base → `coef` vaut toujours `autre` (3.0). Pour corriger : ajouter `materiau` (via la catégorie) dans le `SELECT` de la fonction.

C3. Seuil / délai (ex : nettoyage des box réservées)

- **Fichier** : `API/bdd/boxReq.go` , `GarbageCollectBox` : `INTERVAL 2 DAY` . Changer la valeur pour ajuster le délai.

D. Protéger une route par rôle

- **Middlewares** : `API/auth/jwt.go`
- `VerifyTokenMiddleware(next)` → exige un JWT valide (header `Authorization: Bearer ...`), met `userID / userRole` dans le `context` .
- `VerifyRoleMiddleware(next, roles...)` → 403 si le rôle n'est pas dans la liste.
- **Application** dans le fichier de route, ex. `API/route/users.go` :

```
// Accessible Salarié ET Admin :
http.HandleFunc("GET /admin/users",
    auth.VerifyRoleMiddleware(admin.GetAllUsers, "Salarié", "Administrateur"))
// Réserver aux Admins uniquement :
http.HandleFunc("GET /admin/users",
    auth.VerifyRoleMiddleware(admin.GetAllUsers, "Administrateur"))
```

- **Test à deux comptes** :

```
# Token admin -> 200
curl -i http://localhost:8081/admin/users -H "Authorization: Bearer $TOKEN_ADMIN"
# Token particulier -> 403 "Accès refusé"
curl -i http://localhost:8081/admin/users -H "Authorization: Bearer $TOKEN_USER"
# Sans token -> 401
curl -i http://localhost:8081/admin/users
```

E. Ajouter une colonne dans un tableau admin

Exemple : afficher l'email dans la liste des utilisateurs

Front — `Frontend/script/admin/user.js` , fonction `afficherPageUsers` : 1. Dans `headerHTML` (bloc `u-thead`), ajouter une cellule d'en-tête :

```
<div>Email</div>
```

1. Dans la boucle `usersPage.forEach` (bloc `u-row`), ajouter la cellule :

```
<div class="u-email">${user.email}</div>
```

1. Si le tableau est en CSS grid, ajuster le nombre de colonnes dans `Frontend/style/admin.css` (`.u-thead` , `.u-row` → `grid-template-columns`).

Test : recharger `/admin/users` , la colonne apparaît. (Aucun rebuild backend si le champ est déjà renvoyé par l'API.)

F. Ajouter un endpoint simple

Exemple complet : `GET /admin/users/count`

1. Route — `API/route/users.go` :

```
http.HandleFunc("OPTIONS /admin/users/count", admin.CountUsers)
http.HandleFunc("GET /admin/users/count", auth.VerifyTokenMiddleware(admin.CountUsers))
```

2. Handler — `API/admin/users.go` :

```
func CountUsers(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Access-Control-Allow-Origin", "")
    w.Header().Set("Access-Control-Allow-Headers", "Content-Type, Authorization")
    if r.Method == "OPTIONS" { w.WriteHeader(http.StatusOK); return }

    n, err := bdd.CountUsers()
    if err != nil {
        http.Error(w, "erreur count", http.StatusInternalServerError)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]int{"count": n})
}
```

3. Requête DB — API/bdd/userReq.go :

```
func CountUsers() (int, error) {
    var n int
    err := Db.QueryRow("SELECT COUNT(*) FROM utilisateur").Scan(&n)
    return n, err
}
```

4. Rebuild + test :

```
docker compose up -d --build backend
curl http://localhost:8081/admin/users/count -H "Authorization: Bearer $TOKEN"
# -> {"count": 16}
```

⚠ **Toujours** ajouter la route `OPTIONS` + les 2 headers CORS, sinon erreur CORS en dev local.

Exemple 2 : GET /admin/stats (mini dashboard)

```
// route
http.HandleFunc("OPTIONS /admin/stats", admin.GetStats)
http.HandleFunc("GET /admin/stats", auth.VerifyRoleMiddleware(admin.GetStats, "Administrateur"))
// handler (API/admin/users.go ou un nouveau fichier)
func GetStats(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Access-Control-Allow-Origin", "")
    w.Header().Set("Access-Control-Allow-Headers", "Content-Type, Authorization")
    if r.Method == "OPTIONS" { w.WriteHeader(http.StatusOK); return }
    var users, annonces int
    bdd.Db.QueryRow("SELECT COUNT(*) FROM utilisateur").Scan(&users)
    bdd.Db.QueryRow("SELECT COUNT(*) FROM annonce").Scan(&annonces)
    json.NewEncoder(w).Encode(map[string]int{"users": users, "annonces": annonces})
}
```

Exemple 3 : PATCH / PUT d'un statut — PUT /admin/annonces/validate/{id} existe déjà

Modèle de handler qui change un statut (déjà dans API/admin/annonce.go) :

```
id, _ := strconv.Atoi(r.PathValue("id"))
bdd.Db.Exec("UPDATE annonce SET statut_validation = 'Validé' WHERE id = ?", id)
```

G. Ajouter une entrée de menu / un lien de navigation

- **HTML** de la page → ajouter dans la `<nav>` : `<a class="nav-link" href="/annonces">Annonces</a>` .
- Utiliser les **URLs propres** (/login , /admin , /annonces ...) — elles sont réécrites par Frontend/nginx.conf .
- Le surlignage du lien actif (admin) est géré par Frontend/script/auth_guard.js (surlignerLienActif).

H. Ajouter une traduction (texte multilingue)

1. **HTML** : mettre `data-i18n="ma.cle"` sur l'élément (le texte par défaut reste le fallback).
2. **Base** : ajouter la clé dans les 2 langues :

```
INSERT INTO translations (lang_code, msg_key, msg_value) VALUES  
( 'fr', 'ma.cle', 'Mon texte'), ( 'en', 'ma.cle', 'My text');
```

1. `translate.js` remplace automatiquement au chargement. **Test** : changer de langue via le sélecteur en bas à droite.
-

I. Checklist « je viens de modifier, ça ne marche pas »

1. Erreur Go ? → `cd API && go build ./...` (lit l'erreur exacte).
 2. Conteneur rebuild ? → `docker compose up -d --build backend|frontend`.
 3. Cache navigateur ? → **Ctrl+Shift+R** (ou DevTools « Désactiver le cache »).
 4. Bon code HTTP ? → F12 → Réseau (401/403/500/404/CORS).
 5. La colonne SQL existe et l'ordre `SELECT / Scan` correspond ?
-

05 — Questions / Réponses techniques (soutenance DEV) — ÉTENDU

70+ questions. Réponses courtes, prêtes à dire. Tous les chemins sont réels.

A. Architecture & choix techniques

1. **Présente ton appli en 30 s.** UpcycleConnect : plateforme d'économie circulaire. Les particuliers donnent/vendent des objets, les pros récupèrent la matière via des conteneurs connectés. 4 rôles, back-office admin, paiement Stripe. Déployée en Docker + Nginx + HTTPS.
2. **Pourquoi séparer front et back ?** Découplage, déploiement indépendant (2 conteneurs), l'API peut servir web ET mobile.
3. **Pourquoi Go ?** Compilé, rapide, binaire unique → image Docker `alpine` légère, typage fort, `net/http` suffisant.
4. **Pourquoi pas de framework front ?** Périmètre gérable en HTML/CSS/JS vanilla ; pas d'étape de build ; Nginx sert directement.
5. **Pourquoi MySQL ?** Données très relationnelles (users, annonces, conteneurs, commandes) → clés étrangères et jointures.
6. **Architecture en couches ?** `route/` (URLs) → `admin/` (handlers) → `bdd/` (SQL) → `models/` (structs). 1 domaine = 1 fichier par couche.
7. **Comment est organisé le code Go ?** Packages : `route`, `admin` (handlers), `bdd`, `models`, `auth`. Point d'entrée `API/main.go`.
8. **Quels design patterns ?** Séparation des responsabilités (handler ≠ accès données), middleware pour l'auth. Pas d'ORM : SQL explicite.
9. **Comment communiquent les conteneurs ?** Réseau Docker `uc_net` ; le backend joint MySQL via le nom de service `mysql1`, le front joint le backend via `backend`.

B. API Go

1. **Point d'entrée ?** `API/main.go` : enregistre tous les `route.RoutesX()` puis `http.ListenAndServe(":8081", nil)`.
2. **Routing ?** `net/http` standard (Go 1.22+) avec patterns `GET /admin/users/{id}`.
3. **Comment ajouter une route ?** `http.HandleFunc("GET /chemin", handler)` dans `API/route/<domaine>.go` (+ `OPTIONS`).
4. **Lire un param d'URL ?** `r.PathValue("id")` ; query : `r.URL.Query().Get("x")` ; body : `json.NewDecoder(r.Body).Decode(&s)`.
5. **Renvoyer du JSON ?** `json.NewEncoder(w).Encode(data)`.
6. **Gérer un upload ?** `r.ParseMultipartForm`, `r.FormFile("image")`, puis `SaveUpload` (`API/admin/upload.go`).
7. **Où est la logique d'upload ?** `API/admin/upload.go` : valide extension + MIME + taille, renomme en UUID, écrit dans `UPLOAD_DIR`.
8. **Comment sont gérées les erreurs ?** `http.Error(w, "msg", http.StatusXXX)` + `log fmt.Println`.
9. **Codes HTTP utilisés ?** 200/201 succès, 400 requête invalide, 401 non authentifié, 403 rôle refusé, 404 introuvable, 500 erreur serveur.

C. Frontend

1. **Comment le front appelle l'API ?** `fetch(\ ${API_BASE_URL}/route`, {headers:{Authorization:"Bearer "+token}})`.`
2. **D'où vient API_BASE_URL ?** `Frontend/script/config.js` : `localhost` + `/Frontend/` → `:8081` ; sinon `origin` + `"/api"`.
3. **Où est le token ?** `localStorage` (`token` , `userRole` , `userId`).
4. **Comment protégez-vous une page côté front ?** `Frontend/script/auth_guard.js` (`checkSession(role)`) redirige si pas de token / mauvais rôle. (Sécurité réelle = côté serveur.)
5. **Multilingue ?** `data-i18n` + `translate.js` charge `GET /api/translations?lang=fr` et remplace les textes.
6. **Thème par rôle ?** Variable CSS `--ac` basculée par une classe `.theme-*` ajoutée selon `userRole` (pages annonces/profil).
7. **Génération PDF côté client ?** `jsPDF` dans `Frontend/script/admin/box.js` (`genererRapportLogistique`).
8. **Pagination ?** Côté client dans `Frontend/script/admin/user.js` (`10 users/page` , `slice`).

D. Authentification & rôles

1. **Où se fait l'authentification ?** `API/admin/users.go` (`Login`) vérifie l'email + le mot de passe puis génère un JWT (`API/auth/jwt.go`).
2. **Type de token ?** JWT HS256, clé secrète `jwtKey` dans `API/auth/jwt.go`, contient `userID` + `role` + expiration.
3. **Comment le token est vérifié ?** `VerifyTokenMiddleware` lit le header `Authorization: Bearer`, parse et valide le JWT.
4. **Comment le rôle est vérifié ?** `VerifyRoleMiddleware(next, roles...)` compare `userRole` (du context) à la liste autorisée.
5. **Pourquoi côté serveur ?** Le front (JS, `localStorage`) est modifiable par l'utilisateur → non fiable.
6. **Les 4 rôles ?** `Utilisateur`, `Pro`, `Salarié`, `Administrateur`.
7. **Que contient le JWT ?** `userID`, `role`, date d'expiration — pas de données sensibles.
8. **Où mettez-vous le rôle après vérif ?** Dans le `context` de la requête (`userRole`), relu par le handler.
9. **Comment gérez-vous l'expiration ?** Le token a une durée ; expiré → 401 → l'utilisateur se reconnecte.
10. **Que se passe-t-il si on modifie le JWT côté client ?** La signature ne correspond plus → rejet (401).

E. Base de données

1. **Comment l'API se connecte ?** `API/bdd/db.go` `NewDB()` → `sql.Open("mysql", DSN)`.
2. **D'où viennent les identifiants DB ?** Variables d'env (`DB_HOST`, `DB_USER` ...) ; défauts en local.
3. **Injection SQL ?** Requêtes paramétrées (`?`) systématiques.
4. **Où est le schéma ?** `db/init.sql` (import auto au 1er `docker compose up`).
5. **Que se passe-t-il si la DB tombe ?** Les requêtes échouent → handlers renvoient 500 + log ; le front affiche « erreur serveur ».
6. **Comment sont liées les tables ?** Clés étrangères (`annonce.id_user` → `utilisateur.id`, `annonce.id_categorie` → `categorie.id`, etc.).
7. **Utilisez-vous un ORM ?** Non, SQL explicite avec `database/sql` — plus lisible et contrôlé.
8. **Comment stockez-vous les images ?** Pas en base : chemin public en base (`annonce.image`), fichier dans le volume `/app/uploads`.
9. **Charset ?** `utf8mb4` (accents/emoji). Import à faire avec `--default-character-set=utf8mb4`.

F. Docker & déploiement

1. **Comment lancer avec Docker ?** `docker compose up -d --build` → `uc_mysql`, `uc_backend`, `uc_frontend`.
2. **Combien de conteneurs ?** 3 (MySQL, backend Go, frontend Nginx).
3. **Différence local/prod ?** `docker-compose.yml` `build local` ; `docker-compose-prod.yml` utilise les images `amelbdj/upcycle-*` de Docker Hub sur la VM.
4. **Comment mettez-vous à jour la prod ?** `docker build` → `docker push :vNN` → sur la VM `docker compose pull && up -d`.
5. **Le Dockerfile backend ?** Multi-stage : `build Go` (`golang:alpine`) → image finale `alpine` avec juste le binaire (léger + sécurisé).
6. **Comment persistent les données ?** Volumes Docker : `mysql_data` (base), `uploads_data` (fichiers).
7. **Rôle de Nginx ?** Sert les fichiers statiques, proxy `/api` → backend, proxy `/uploads`, réécrit les URLs propres.
8. **Comment prouver que ce n'est pas localhost ?** `https://upcycleconnect.pro` (IP publique, certificat HTTPS, Nginx hôte Ubuntu).
9. **Healthcheck ?** MySQL a un healthcheck ; le backend attend `service_healthy` (`depends_on`).

G. Sécurité & validations

1. **Mots de passe ?** Hashés `bcrypt` (`golang.org/x/crypto/bcrypt`).
2. **Validez-vous les entrées ?** Uploads (ext/MIME/taille), champs requis côté handler, requêtes paramétrées.
3. **CORS ?** Headers `Access-Control-*` + réponses `OPTIONS` sur chaque handler.
4. **Secrets (Stripe/JWT) ?** En variables d'environnement (fallback en dur dans le code pour le dev). À sortir du code en prod.
5. **Un particulier peut-il accéder à l'admin ?** Non : `VerifyRoleMiddleware("Administrateur")` renvoie 403.
6. **Protégez-vous les fichiers uploadés ?** Servis en lecture via `/uploads` ; noms UUID (non devinables).

Base de données

```
# Ouvrir un shell MySQL (local)
docker exec -it uc_mysql mysql -uupcycle -pupcyclePass123 pa2026

# Requête rapide
docker exec uc_mysql mysql -uupcycle -pupcyclePass123 pa2026 -e "SHOW TABLES;"

# EXPORTER la base (dump)
docker exec uc_mysql mysqldump -uupcycle -pupcyclePass123 pa2026 > sauvegarde_pa2026.sql

# IMPORTER un dump (⚠ utf8mb4 pour garder les accents)
docker exec -i uc_mysql mysql --default-character-set=utf8mb4 -uupcycle -pupcyclePass123 pa2026 < db/init.sql

# Reset complet de la base (réimport init.sql au prochain up)
docker compose down -v && docker compose up -d
```

Vérifs rapides (santé)

```
# L'API répond ? (401 = vivante, protégée)
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8081/admin/users

# Le front répond ?
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8088/

# Ports occupés (Windows / Git Bash)
netstat -ano | grep ':8081'
netstat -ano | grep ':8088'

# Variables d'env vues par le backend
docker exec uc_backend env | grep -E "DB_|STRIPE|ONESIGNAL|UPLOAD"
```

Tests

```
# Pas de tests Go automatisés dans le projet (à confirmer)
cd API && go build ./...      # au minimum : vérifie que ça compile
cd API && go vet ./...        # analyse statique
node -c Frontend/script/xxx.js # vérifie la syntaxe d'un fichier JS
```

Git (sauvegarde soutenance)

```
git status
git add -A
git commit -m "Sauvegarde avant soutenance"

# Branche de secours pour les mods en direct
git checkout -b soutenance-modifs
# ... mods ...
git checkout amel      # revenir à la branche stable si besoin
```

Docker Hub (mise à jour prod)

```
docker build -t amelbdj/upcycle-backend:vNN ./API
docker build -t amelbdj/upcycle-frontend:vNN ./Frontend
docker push amelbdj/upcycle-backend:vNN
docker push amelbdj/upcycle-frontend:vNN
# Sur la VM : éditer docker-compose-prod.yml (tags) puis :
docker compose pull && docker compose up -d
```

07 — Debug d'urgence (soutenance)

Réflexe n°1 : `docker ps` (tout tourne ?) puis `docker compose logs -f backend`.

#	Problème	Symptôme	Cause probable	Fichier à vérifier	Commande	Correction rapide
1	Front ne démarre pas	Conteneur <code>uc_frontend</code> exit	Erreur Nginx / backend introuvable en upstream	<code>Frontend/nginx.conf</code>	<code>docker compose logs frontend</code>	Lancer la stack complète (<code>docker compose up -d</code>) — le front seul échoue car <code>proxy_pass</code> backend
2	API ne démarre pas	<code>uc_backend</code> redémarre en boucle	Erreur de compilation / port pris / DB absente	logs backend	<code>docker compose logs -f backend</code>	Corriger l'erreur Go ; vérifier <code>mysql healthy</code> avant
3	DB ne répond pas	500 sur toutes les routes	MySQL pas prêt / mauvais identifiants	<code>API/bdd/db.go</code> , <code>docker-compose.yml</code> (DB_*)	<code>docker compose logs mysql ;</code> <code>docker exec uc_mysql mysqladmin ping -uroot -p...</code>	Attendre <code>healthy</code> ; vérifier <code>DB_HOST=mysql</code>
4	Erreur CORS	Console: <i>blocked by CORS policy</i>	Handler sans <code>OPTIONS</code> ou sans header <code>Authorization</code> autorisé	<code>API/admin/<domaine>.go</code>	—	Ajouter <code>Access-Control-Allow-Headers: Content-Type, Authorization</code> + route <code>OPTIONS</code> . N'arrive pas en prod (même origine via <code>/api</code>)
5	401 Unauthorized	Appels rejetés	Token absent/expiré	<code>API/auth/jwt.go</code> , <code>localStorage</code>	Console : <code>localStorage.getItem('token')</code>	Se reconnecter (le token est régénéré au login)
6	403 Forbidden	<i>Accès refusé...</i>	Mauvais rôle	<code>API/auth/jwt.go</code> (<code>VerifyRoleMiddleware</code>)	<code>localStorage.getItem('userRole')</code>	Utiliser un compte du bon rôle
7	500 Internal	Réponse 500	Erreur SQL / scan / nil	logs backend + <code>API/bdd/*Req.go</code>	<code>docker compose logs -f backend</code>	Lire la ligne d'erreur Go (nom de colonne, table...)
8	Docker ne build pas	<code>docker build</code> échoue	Erreur Go / fichier manquant	<code>API/Dockerfile</code> , <code>go.mod</code>	<code>cd API && go build ./...</code>	Corriger l'erreur Go affichée avant de rebuild
9	Variable d'env manquante	Stripe/OneSignal KO, DB par défaut	<code>.env</code> absent	<code>.env</code> , <code>docker-compose.yml</code>	<code>docker exec uc_backend env grep DB_</code>	Copier <code>.env.example</code> → <code>.env</code> et remplir
10	Endpoint introuvable (404)	404 sur une route	Route non déclarée / faute de frappe	<code>API/route/*.go</code> + <code>API/main.go</code>	<code>grep -r "chemin" API/route/</code>	Vérifier la déclaration <code>HandleFunc</code> et que <code>RoutesX()</code> est appelé dans <code>main.go</code>
11	Modif faite mais invisible	Rien ne change à l'écran	Cache navigateur ou conteneur pas rebuild	—	—	Ctrl+Shift+R ; sinon <code>docker compose up -d --build frontend</code>
12	Accents cassés (« ? »)	Textes avec ?	Import DB en latin1	<code>db/init.sql</code> , table <code>translations</code>	—	Réimporter avec <code>--default-character-set=utf8mb4</code>
13	413 Request Too Large	Upload image échoue	Limite Nginx (hôte VM)	<code>Frontend/nginx.conf</code> (conteneur OK 30M) / Nginx hôte	—	Ajouter <code>client_max_body_size 30M</code> ; dans le Nginx hôte de la VM
14	Paiement « impossible de contacter le serveur »	Erreur au checkout event	Salarié sans <code>stripe_account_id</code>	<code>API/admin/stripe.go</code> , table utilisateur	<code>SELECT stripe_account_id FROM utilisateur WHERE id=..</code>	Renseigner un <code>stripe_account_id</code> de test

Le réflexe « ça marche pas » en 4 étapes

1. `docker ps` → les 3 conteneurs sont-ils **Up** (et mysql **healthy**) ?
2. `docker compose logs -f backend` → lire la vraie erreur.
3. Console navigateur (F12) → onglet **Réseau** : quel code HTTP ? (401/403/500/CORS/404)
4. Reproduire l'appel en `curl` pour isoler front vs back.

08 — Base de données (pa2026)

SGBD : MySQL 8. Schéma + données : `db/init.sql` (import auto au 1er `docker compose up`). Dump de référence : `pa2026.sql`. Connexion locale :
`docker exec -it uc_mysql mysql -uupcycle -pupcyclePass123 pa2026`.

Tables principales (31 au total)

Table	Rôle	Colonnes importantes
utilisateur	comptes	<code>id</code> , <code>nom</code> , <code>prenom</code> , <code>email</code> , <code>mot_de_passe</code> (bcrypt), <code>role</code> (enum), <code>validation</code> , <code>score</code> , <code>stripe_account_id</code> , <code>est_premium</code> , <code>plan_abo</code>
annonce	dons/ventes	<code>id</code> , <code>id_user</code> , <code>id_categorie</code> , <code>titre</code> , <code>description</code> , <code>type</code> , <code>prix</code> , <code>statut_validation</code> , <code>etat</code> , <code>poids_kg</code> , <code>ville</code> , <code>code_postal</code> , <code>image</code> , <code>is_sponsored</code> , <code>statut_vente</code> , <code>id_box</code> , <code>created_at</code>
categorie	matériaux	<code>id</code> , <code>libelle</code> (Textile, Bois, Plastique, Métal)
conteneur	points de collecte	<code>id</code> , <code>nom</code> , <code>adresse</code>
box / box_conteneur	casiers	<code>id</code> , <code>numero</code> , <code>statut</code> , <code>taille</code> , <code>id_conteneur</code>
depot_box / historique_conteneurs	dépôts/retraits	réservation, dépôt, retrait, dates, codes
order	commandes	<code>id</code> , <code>acheteur</code> , <code>annonce</code> , <code>montant</code> , <code>commission</code> , <code>statut</code>
paiement	paiements Stripe	<code>montant</code> , <code>statut</code>
document	factures/contrats PDF	<code>id_document</code> , <code>id_user</code> , <code>type_doc</code> , <code>url_pdf</code> , <code>id_commande</code>
documents_legaux	pièces justificatives users	<code>user_id</code> , <code>chemin_fichier</code>
evenement	events/formations	<code>id</code> , <code>id_salarie</code> , <code>titre</code> , <code>type</code> , <code>prix</code> , <code>date_debut</code> , <code>lieu</code> , <code>nb_places</code> , <code>image_url</code> , <code>statut_validation</code>
inscription	inscriptions events	<code>user</code> , <code>event</code>
ressource_pedagogique	PDF de formation	<code>url_fichier</code>
article_news	articles	<code>id_article</code> , <code>id_salarie</code> , <code>titre</code> , <code>contenu</code> , <code>image_url</code> , <code>statut</code>
topic_forum / message_forum	forum	<code>id_topic</code> , <code>titre</code> / <code>id_message</code> , <code>contenu</code> , <code>est_modere</code> , <code>est_signale</code>
message	messaging chat	<code>expéditeur</code> , <code>destinataire</code> , <code>contenu</code> , <code>lu</code>
notification	notifs in-app	<code>user</code> , <code>message</code> , <code>est_lu</code>
abonnement / plan_abo	abonnements pro	<code>plan</code> , <code>statut</code>
projet_pro / etapes_projet	projets pro	<code>avant/après</code> , <code>CO2</code>
translations / languages	multilingue	<code>lang_code</code> , <code>msg_key</code> , <code>msg_value</code>
upcycling_score	score éco	historique de points
log_connexion	logs auth	<code>ip</code> , <code>date</code>

Table	Rôle	Colonnes importantes
campagne_pub , partenaire , dictionnaire , plan_abo	annexes	—

Relations / clés étrangères principales

```

utilisateur (1) —< annonce (id_user)
categorie (1) —< annonce (id_categorie)
utilisateur (1) —< evenement (id_salarie)
evenement (1) —< inscription >— (1) utilisateur
conteneur (1) —< box (id_conteneur)
annonce (1) —< order >— (1) utilisateur (acheteur)
utilisateur (1) —< document (id_user)
topic_forum (1) —< message_forum (id_topic)
utilisateur (1) —< notification

```

Requêtes SQL utiles

```

-- Voir tous les utilisateurs par rôle
SELECT id, prenom, nom, email, role, validation FROM utilisateur ORDER BY role;

-- Annonces en attente de validation
SELECT id, titre, prix, statut_validation FROM annonce WHERE statut_validation='En attente';

-- Catégories
SELECT * FROM categorie;

-- Conteneurs + nb de casiers
SELECT c.nom, c.adresse, COUNT(b.id) AS nb_box
FROM conteneur c LEFT JOIN box b ON b.id_conteneur=c.id GROUP BY c.id;

-- Score d'un utilisateur
SELECT id, prenom, score FROM utilisateur WHERE id=1;

-- Rendre visibles tous les messages du forum
UPDATE message_forum SET est_moderé=0;

-- Donner un compte Stripe test à un salarié
UPDATE utilisateur SET stripe_account_id='acct_...' WHERE id=40;

```

Import / export

```

# Export (sauvegarde)
docker exec uc_mysql mysqldump -uupcycle -pupcyclePass123 pa2026 > backup.sql

# Import (⚠ utf8mb4 pour les accents)
docker exec -i uc_mysql mysql --default-character-set=utf8mb4 -uupcycle -pupcyclePass123 pa2026 < db/init.sql

```

Générer une base vide vs remplie

- **Remplie** (démon) : `db/init.sql` contient schéma + **données** → import automatique au 1er démarrage Docker, ou manuel (commande ci-dessus).
- **Vide** : importer uniquement les `CREATE TABLE` (retirer les `INSERT`), ou : `sql -- vider une table sans supprimer la structure TRUNCATE TABLE annonce;`

Seed

- **Script de seed dédié : non trouvé** (`db/seed.*` inexistant). Le « seed » = les `INSERT` présents dans `db/init.sql`.

- **Plan proposé** (si demandé, à faire seulement sur validation) : créer `db/seed.sql` avec 1 compte par rôle (mot de passe bcrypt), 3–4 annonces, 1 conteneur + casiers, 1 événement, quelques catégories.

09 — Comptes de test

Les mots de passe sont **hashés (bcrypt)** en base : impossibles à extraire. Les comptes ci-dessous existent réellement dans `pa2026` (table `utilisateur`). **Mot de passe : à confirmer** (celui que tu as défini au seed / que tu connais). Note-le dans la colonne prévue avant la soutenance.

Comptes existants (validés, prêts pour la démo)

Rôle	Email	Statut	Mot de passe (à remplir)
Administrateur	<code>test.admin@renova.test</code>	Validé	_____
Administrateur	<code>test.admin@test.fr</code>	Validé	_____
Administrateur (réel)	<code>amelbdj213@gmail.com</code>	Validé	_____
Salarié	<code>test.salarie@renova.test</code>	Validé	_____
Salarié	<code>test.salarie@test.fr</code>	Validé	_____
Pro	<code>test.pro@renova.test</code>	Validé	_____
Pro	<code>test.pro@test.fr</code>	Validé	_____
Utilisateur (particulier)	<code>test.client@renova.test</code>	Validé	_____
Utilisateur	<code>test.user@test.fr</code>	Validé	_____

Vérifier la liste à jour : `SELECT email, role, validation FROM utilisateur WHERE validation='Validé';`

Usage en démo (parcours par compte)

Administrateur (`test.admin@renova.test`) → redirige vers `/admin`

- Vue d'ensemble (KPI, activité récente dynamique, badge validations).
- **Utilisateurs** (`/admin/users`) : pagination 10/page, valider/refuser/bannir.
- **Validations** (`/admin/validations`) : approuver une annonce/un event → il disparaît en direct.
- **Finances** (`/admin/finances`), **Documents** (PDF), **Conteneurs** (`/admin/conteneurs` + bouton **Rapport logistique PDF**).

Salarié (`test.salarie@renova.test`) → `/salarie`

- Créer un **événement/formation** (⚠ nécessite `stripe_account_id`), planning, forum, articles.

Pro (`test.pro@renova.test`) → `/pro`

- Espace pro **en teal**, abonnements, projets avant/après, sponsoriser une annonce.
- Page annonces/profil aux **couleurs pro**.

Utilisateur (`test.client@renova.test`) → `/client`

- Créer une **annonce** (don/vente), voir le tableau de bord (Annonces, Dépôt actif, Score éco — tous dynamiques), déposer/récupérer en conteneur (simulateur).

Si tu veux (re)créer des comptes propres

- Via l'inscription : page `/register` (rôle Utilisateur/Pro).

- Via l'admin : `/admin/users` → « Ajouter » (permet de choisir le rôle).
- Un compte créé peut être **En attente** → le valider depuis l'admin avant la démo.

10 — Script de démonstration DEV (10–15 min)

Ton objectif : montrer une plateforme **fonctionnelle, déployée en ligne, multi-rôles**. Rester fluide, orienté produit. Avoir 2 onglets ouverts : **prod** `https://upcycleconnect.pro` (preuve non-localhost) et **local** `http://localhost:8088` (pour la modif en direct).

Avant de commencer (30 s)

- « L'application s'appelle **UpcycleConnect**, une plateforme d'économie circulaire : les particuliers donnent ou vendent des objets, les professionnels récupèrent la matière, le tout via des **conteneurs connectés**. »
- « Elle est **déployée en ligne** sur `https://upcycleconnect.pro`, en Docker derrière Nginx et HTTPS. »

Déroulé

1. Page d'accueil + connexion (1 min)

- Ouvrir `https://upcycleconnect.pro/` → landing.
- Montrer le **sélecteur de langue** (multilingue) et l'URL propre (`/login`, `/annonces` ...).
- Se connecter en **Utilisateur** (`test.client@renova.test`).

2. Espace particulier (2 min) — compte Utilisateur

- Tableau de bord : **stats dynamiques** (Annonces, Dépôt actif, **Score éco**).
- Créer une **annonce** (titre, catégorie, prix/don, photo) → « soumise à validation ».
- Aller sur `/annonces` : filtre **catégories depuis la base**, recherche, tri.

3. Espace professionnel (2 min) — compte Pro

- Se connecter Pro (`test.pro@renova.test`) → `/pro` (interface **teal**).
- Montrer abonnement Premium, projets **avant/après** avec impact CO₂.
- Ouvrir `/annonces` : mêmes annonces mais **aux couleurs pro** (thème adaptatif).

4. Back-office admin (3 min) — compte Admin ★ point fort

- Se connecter Admin (`test.admin@renova.test`) → `/admin`.
- **Vue d'ensemble** : KPI (utilisateurs, revenus, conteneurs), **activité récente dynamique**, badge de validations.
- **Validations** : approuver l'annonce créée à l'étape 2 → elle disparaît **en direct**.
- **Utilisateurs** (`/admin/users`) : **pagination**, validation/refus d'un compte.
- **Conteneurs** (`/admin/conteneurs`) : cliquer un conteneur → casiers, puis « **Rapport logistique** » → **téléchargement d'un PDF** généré.
- **Documents** : factures/contrats PDF.

5. Preuve technique : déploiement + API (2 min)

- Montrer un terminal : `docker ps` (3 conteneurs Up), `docker compose logs backend`.
- Montrer un appel API en direct (Postman/curl) : `GET /admin/users` avec le token.
- « L'API Go tourne sur `:8081`, protégée par **JWT** et vérification de **rôle côté serveur**. »

6. Modification en direct (si demandée, 2–3 min)

- Prendre un exercice simple préparé (voir `12_EXERCICES`), ex : changer le **taux de commission** dans `API/admin/stripe.go`, ou ajouter une **colonne** dans le tableau admin.
- `docker compose up -d --build backend` (ou front) → Ctrl+Shift+R → montrer le résultat.

À dire à l'oral (phrases prêtes)

- « Front et back sont **séparés** et déployés en deux conteneurs distincts. »
- « Toute action sensible est **vérifiée côté serveur** : le front n'est jamais une source de confiance. »
- « Les fichiers uploadés sont stockés dans un **volume Docker persistant**, jamais en base. »

À ÉVITER de montrer

- Le flux de **paiement Stripe complet** (dépend de comptes Connect + webhook — risque de blocage live). Dire « le paiement passe par Stripe Checkout + Connect, avec commission de 5 % ».
- Le **push OneSignal** en local (désactivé sur localhost, HTTPS requis).
- Les pages/tables de données incohérentes ou de test brut (comptes « Rejeté », données `azerty...`).
- Ouvrir la console avec des erreurs CORS **en local** (n'existent pas en prod) : démontrer plutôt sur la prod.

11 — Checklist avant soutenance

La veille

- [] `git add -A && git commit -m "Sauvegarde avant soutenance"` (sauvegarde du projet).
- [] Créer la branche de secours : `git checkout -b soutenance-modifs` puis revenir sur `amel`.
- [] Exporter la base : `docker exec uc_mysql mysqldump -uupcycle -pupcyclePass123 pa2026 > backup_soutenance.sql`.
- [] Vérifier que la **prod** est à jour (images `frontend:v32` / `backend:v8` déployées) : `docker compose -f docker-compose-prod.yml pull && up -d` sur la VM.
- [] Générer le **PDF du kit** (voir `KIT_SOUTENANCE_DEV_COMPLET.md`) et l'avoir hors-ligne.
- [] Faire des **captures d'écran de secours** de chaque page clé (au cas où le réseau tombe).

Le jour J (30 min avant)

- [] **Projet lancé** : `docker compose up -d` → `docker ps` montre 3 conteneurs **Up**.
- [] **MySQL healthy** : `docker ps` (statut `healthy`).
- [] **API OK** : `curl -s -o /dev/null -w "%{http_code}" http://localhost:8081/admin/users` → `401`.
- [] **Front OK** : ouvrir `http://localhost:8088/` → landing s'affiche.
- [] **Base OK** : `docker exec uc_mysql mysql -uupcycle -pupcyclePass123 pa2026 -e "SELECT COUNT(*) FROM utilisateur;"`.
- [] **Comptes de test OK** : se connecter avec 1 compte de chaque rôle (voir `09_COMPTE_TEST.md`, mots de passe remplis).
- [] **Site externe accessible** : ouvrir `https://upcycleconnect.pro` (preuve non-localhost).
- [] **Postman/curl prêt** : collection avec login + 2-3 routes protégées + le token en variable.
- [] **Exports DB prêts** : `backup_soutenance.sql` accessible.
- [] **Cache navigateur** : ouvrir DevTools → « Désactiver le cache » pour les modifs en direct.

Onglets à préparer

- [] `https://upcycleconnect.pro/` (prod)
- [] `http://localhost:8088/` (local, pour modif)
- [] Un terminal à la racine du projet
- [] L'éditeur de code ouvert sur `API/` et `Frontend/script/`
- [] Ce kit (PDF) ouvert

Réflexes anti-panique

- [] Savoir dire où se trouve **chaque fichier** (voir `01_ARCHITECTURE`).
- [] Avoir répété **2-3 exercices de modif** (voir `12_EXERCICES`).
- [] Connaître les 4 commandes clés : `docker compose up -d --build`, `docker compose logs -f backend`, `docker ps`, `docker exec -it uc_mysql mysql ...`.

12 — Exercices d'entraînement (modif en direct) — ÉTENDU

18 exercices chronométrés. Fais-les 2-3 fois jusqu'à être fluide. Après chaque modif : rebuild du conteneur concerné + Ctrl+Shift+R. Token de test : voir 04_GUIDE_MODIFICATIONS_DIRECT.md (section 0).

FRONTEND

F1 — Ajouter une colonne « Email » dans le tableau admin (🕒 5 min)

- **Objectif** : afficher l'email dans la liste des utilisateurs.
- **Fichier** : `Frontend/script/admin/user.js` (`afficherPageUsers`).
- **Solution** : ajouter `<div>Email</div>` dans `headerHTML` (bloc `u-thead`) et `<div class="u-email">${user.email}</div>` dans la ligne `u-row`.
- **Vérif** : recharger `/admin/users` → la colonne apparaît.

F2 — Changer la couleur d'accent d'une page (🕒 4 min)

- **Objectif** : changer la couleur du thème annonces.
- **Fichier** : `Frontend/style/annonceAll.css` (`:root { --ac: ... }`).
- **Solution** : `--ac: 255,120,0;` (orange).
- **Vérif** : recharger `/annonces`, les accents/bordures changent.

F3 — Ajouter un champ « ville » au formulaire d'annonce (🕒 6 min)

- **Objectif** : envoyer la ville avec l'annonce.
- **Fichiers** : la page du formulaire (HTML) + le JS qui construit le `FormData`.
- **Solution** : `<input id="ville">` + `formData.append("ville", document.getElementById("ville").value)`.
- **Vérif** : F12 → Réseau → la requête contient bien `ville`.

F4 — Ajouter un lien dans la barre de navigation (🕒 3 min)

- **Objectif** : ajouter un lien « À propos » dans une nav.
- **Fichier** : une page HTML (bloc `<nav>`).
- **Solution** : `<a class="nav-link" href="/a-propos">À propos</a>` (URL propre, gérée par Nginx).
- **Vérif** : cliquer → la page `/a-propos` s'ouvre.

F5 — Afficher un compteur dynamique (🕒 6 min)

- **Objectif** : afficher le nombre d'annonces trouvées.
- **Fichier** : `Frontend/script/annonces/annonceAll.js` (`displayAnnonces`).
- **Solution** : `document.getElementById("resultCount").textContent = items.length;` (déjà présent → variante : ajouter un `console.log` ou un autre élément).
- **Vérif** : le compteur reflète le filtre.

F6 — Ajouter un bouton de tri (🕒 5 min)

- **Objectif** : trier les annonces par prix décroissant.
- **Fichier** : `Frontend/script/annonces/annonceAll.js` (`sortListings` existe déjà).
- **Solution** : appeler `sortListings('price-desc')` depuis un bouton.
- **Vérif** : l'ordre change à l'écran.

API GO

A1 — Endpoint `GET /admin/users/count` (🕒 8 min)

- **Fichiers** : `API/route/users.go` + `API/admin/users.go` + `API/bdd/userReq.go`.

- **Solution** : code complet dans `04_GUIDE_MODIFICATIONS_DIRECT.md` (section F).
- **Vérif** : `curl http://localhost:8081/admin/users/count -H "Authorization: Bearer $TOKEN" → {"count":N}`.

A2 — Modifier la commission (5 % → 10 %) (🕒 4 min)

- **Fichier** : `API/admin/stripe.go`.
- **Solution** : `(unitAmount * 5)/100 → * 10`, et `0.05 → 0.10`.
- **Vérif** : `cd API && go build ./...` OK, puis `GET /admin/finance/overview`.

A3 — Réserver `/admin/users` aux Admins (🕒 5 min)

- **Fichier** : `API/route/users.go`.
- **Solution** : `auth.VerifyRoleMiddleware(admin.GetAllUsers, "Administrateur")`.
- **Vérif** : token Admin → 200 ; token Salarié → 403.

A4 — Ajouter un champ à une struct + le renvoyer (🕒 8 min)

- **Objectif** : renvoyer le `telephone` de l'utilisateur.
- **Fichiers** : `API/models/users.go` (struct), `API/bdd/userReq.go` (SELECT + Scan).
- **Solution** : ajouter `Telephone string \ json:"telephone"` + `COALESCE(telephone,")` dans le SELECT + `&user.Telephone`` dans le Scan.
- **Vérif** : `GET /admin/users/1` renvoie le champ `telephone`.

A5 — Endpoint `GET /admin/stats` (🕒 8 min)

- **Objectif** : renvoyer `{users, annonces}`.
- **Fichiers** : `API/route/divers.go` (ou `users.go`) + handler.
- **Solution** : voir `04_GUIDE` section F, exemple 2.
- **Vérif** : `curl http://localhost:8081/admin/stats -H "Authorization: Bearer $TOKEN"`.

A6 — Ajouter une validation d'entrée (🕒 6 min)

- **Objectif** : refuser une annonce sans titre.
- **Fichier** : `API/admin/annonce.go` (`CreateAnnonce`).
- **Solution** :

```
if r.FormValue("titre") == "" {
    http.Error(w, "Le titre est obligatoire", http.StatusBadRequest)
    return
}
```

- **Vérif** : POST sans titre → 400.

BASE DE DONNÉES

D1 — Ajouter une colonne (🕒 5 min)

- **Objectif** : `telephone` sur `utilisateur`.
- **Commande** : `ALTER TABLE utilisateur ADD COLUMN telephone VARCHAR(20) NULL;`
- **Vérif** : `DESCRIBE utilisateur;`

D2 — Requête d'analyse (🕒 4 min)

- **Objectif** : nombre d'annonces par catégorie.
- **Solution** :

```
SELECT c.libelle, COUNT(a.id) nb FROM categorie c
LEFT JOIN annonce a ON a.id_categorie=c.id GROUP BY c.id;
```

- **Vérif** : résultat affiché.

D3 — Modifier une donnée métier (🕒 4 min)

- **Objectif** : valider un utilisateur en attente.
- **Solution** : `UPDATE utilisateur SET validation='Validé' WHERE id=42;`
- **Vérif** : le compte peut se connecter sans être bloqué.

D4 — Export / réimport (🕒 5 min)

- **Objectif** : sauvegarder puis restaurer la base.
- **Solution** :

```
docker exec uc_mysql mysqldump -uupcycle -pupcyclePass123 pa2026 > b.sql
docker exec -i uc_mysql mysql --default-character-set=utf8mb4 -uupcycle -pupcyclePass123 pa2026 < b.sql
```

- **Vérif** : `SELECT COUNT(*) FROM utilisateur;` identique.

DOCKER / DEBUG

T1 — Rebuild + logs d'un service (🕒 3 min)

- **Commandes** : `docker compose up -d --build backend` puis `docker compose logs -f backend`.
- **Vérif** : voir « Connexion à la bdd reussie ».

T2 — Diagnostiquer un 500 (🕒 5 min)

- **Méthode** : provoquer une erreur (mauvais param), lire les logs backend, nommer la cause (colonne, scan, nil).
- **Vérif** : savoir expliquer l'erreur.

T3 — Vérifier ports + variables d'env (🕒 3 min)

- **Commandes** :

```
netstat -ano | grep ':8081'
docker exec uc_backend env | grep -E "DB_|STRIPE|UPLOAD"
```

- **Vérif** : le port 8081 écoute, les variables sont présentes.

Plan d'entraînement conseillé (la veille, 45 min)

1. F1 + F3 (colonne + champ formulaire) — les plus probables.
2. A1 + A2 (endpoint + règle métier) — les plus impressionnants.
3. A3 (protection par rôle) — la question sécurité classique.
4. D1 + T1 (colonne DB + rebuild/logs) — le réflexe complet.